

AI Agents

From chatbot thinking to multi-agent workflows

A practical workshop on how agents help technical and non-technical teams turn ideas into real outputs.

Developers

Managers

Multi-agent

Context-aware

Cost-aware

For developers, managers, and technical & non-technical participants

Chatbot

Assistant

RAG

Agent

Multi-agent

Sponsors & community support

Recognising the organisations behind this workshop

Main sponsor

Optum / UHC Tech

Supporting the workshop and enabling practical AI learning for teams.

Community representation

Global AI Community Letterkenny

Connecting local participants to a wider AI learning community.

Why this matters

This workshop combines enterprise perspective + community learning to show how agentic workflows can be applied in real work.

About the speaker

Sathish Kumar Krisnamoorthy

Senior Software Engineer
Optum Letterkenny

Focus areas: Software Engineering, Cloud, Data Engineering, and AI LLM technology.

Software Engineer

Cloud & Automation

Data Engineer

Workshop links

https://github.com/raasalgul/workshop_agentic_sde_workflow

https://github.com/raasalgul/workshop_agentic_manager_workflow

<http://globalai.community/eventbadge/116359>



Workshop objective

Show how multi-agent workflows help both technical and non-technical participants

For developers

Use agents to clarify requirements, build software, organise work, and accelerate delivery.

For managers

Use agents to turn meeting transcripts and unstructured input into structured actions and documents.

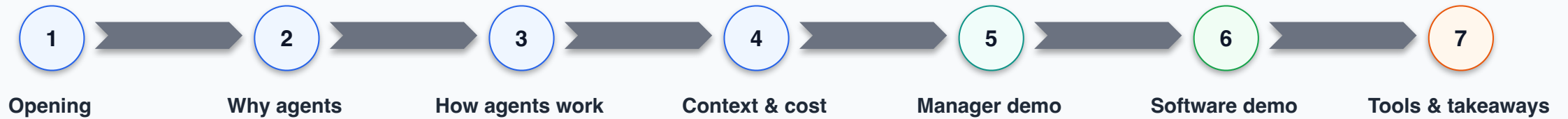
What participants will see

Two live-style examples:

- Business / manager workflow
- Software product workflow

Workshop flow

One agenda. One clear story.

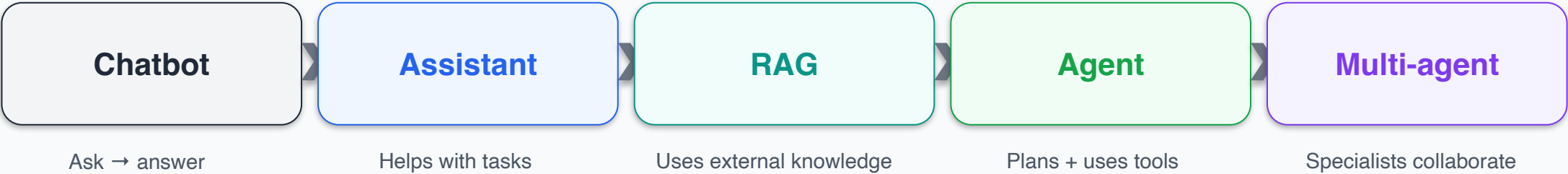


Story of the workshop

How AI evolved → how agents work → what they need → how to control cost → two practical workflow demos.

From chatbot to multi-agent workflows

A quick view of how AI usage has evolved

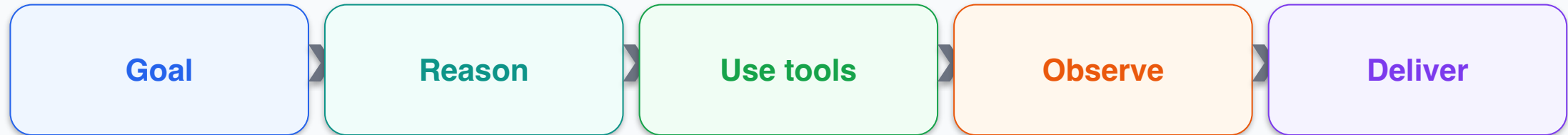


Key idea

The workshop focus is the last step: multiple agents working together to deliver real outcomes.

What is an AI agent?

An agent works toward a goal instead of only answering a prompt



Assistant mindset

“Here is an answer.”

Agent mindset

“I used instructions, tools, and context to produce the output.”

Agent building blocks

The core pieces behind a useful agent system

Instructions

How should I behave?

Skills

What procedure can I follow?

Knowledge

What project or domain info do I know?

Tools

What systems can I access?

Model

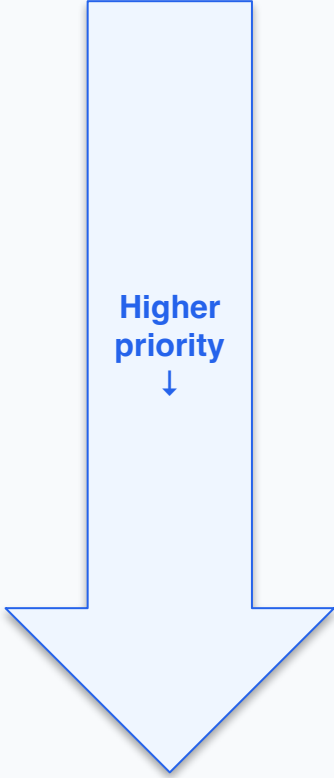
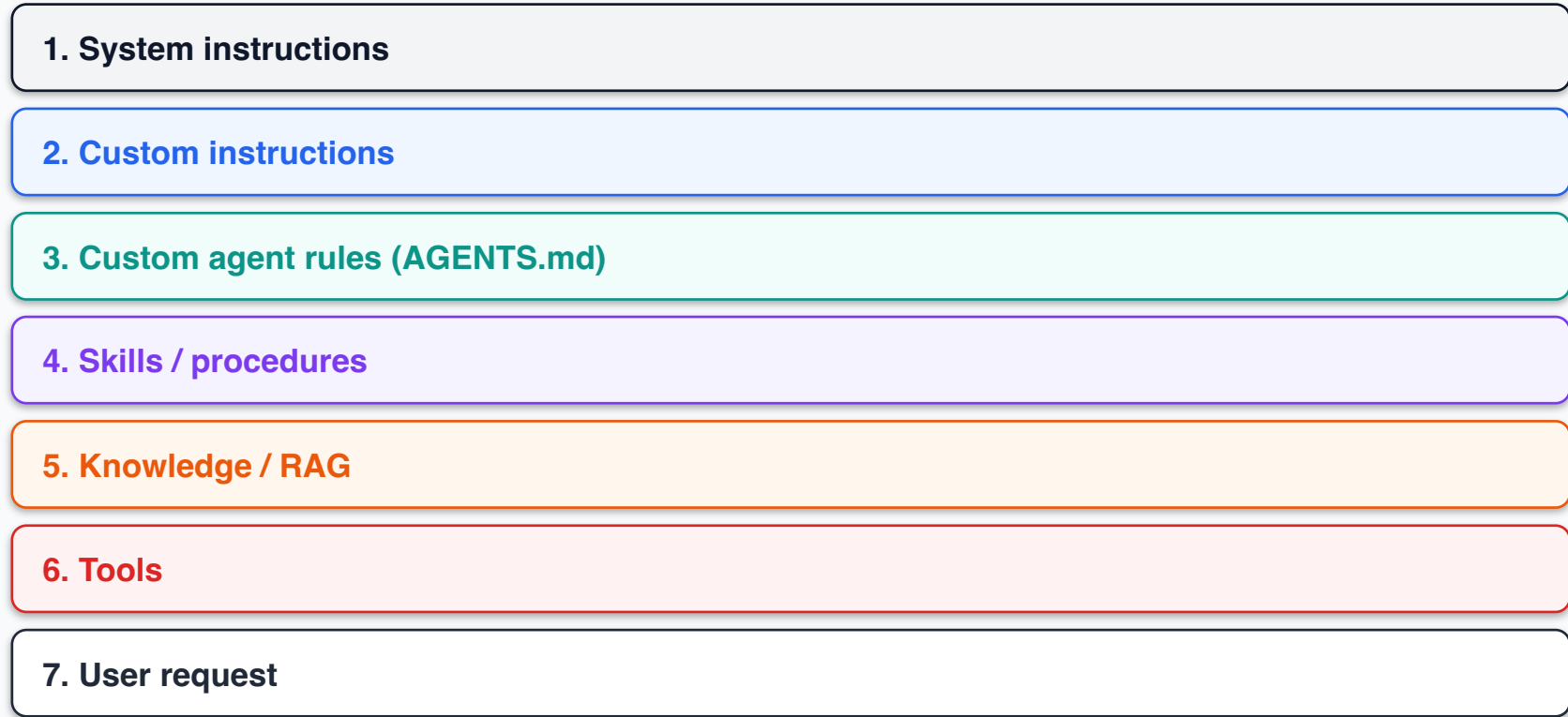
What intelligence powers me?

Guardrails

What must I avoid or escalate?

Instruction hierarchy

Not all instructions have the same priority or scope



Runtime assembly

The model sees only the relevant instructions, files, retrieved context, tool results, and the current request.

Why multi-agent workflows?

Break larger work into specialist roles with clear handoffs

Specialists

Each agent focuses on a specific job, such as planning, coding, summarising, or reviewing.

Better handoffs

Outputs become structured inputs for the next step in the workflow.

Clearer control

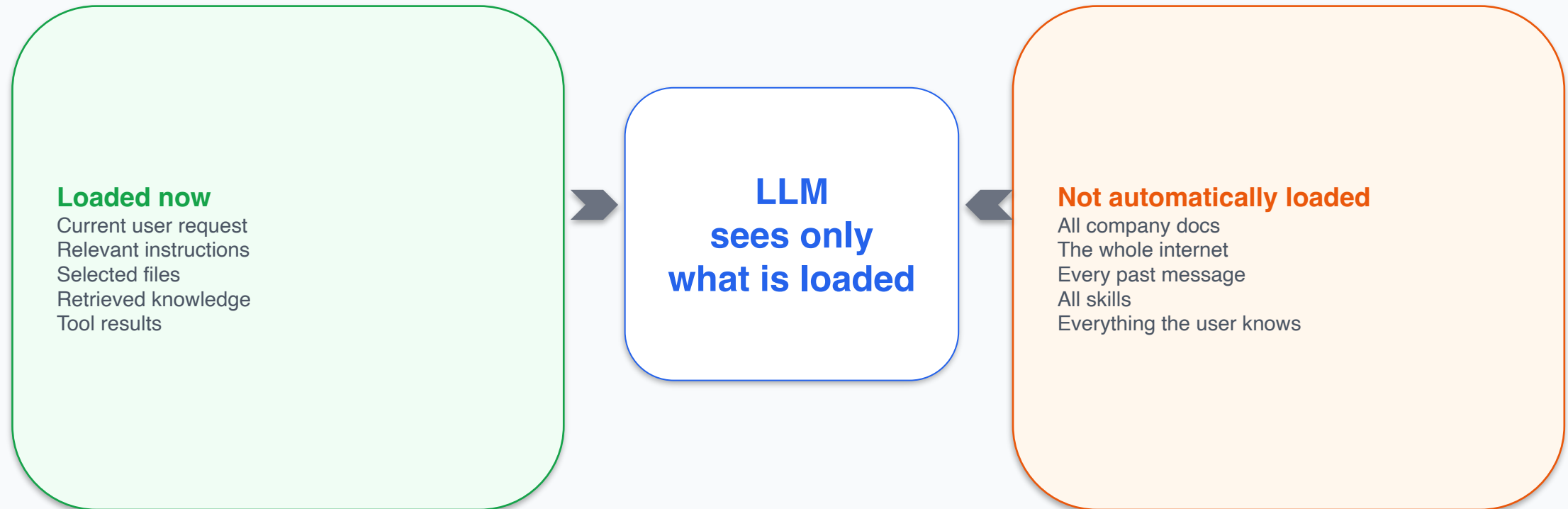
You can tune prompts, tools, and cost by role instead of using one giant agent.

Workshop examples

Manager flow: transcript → business outputs | Software flow: specification → working product

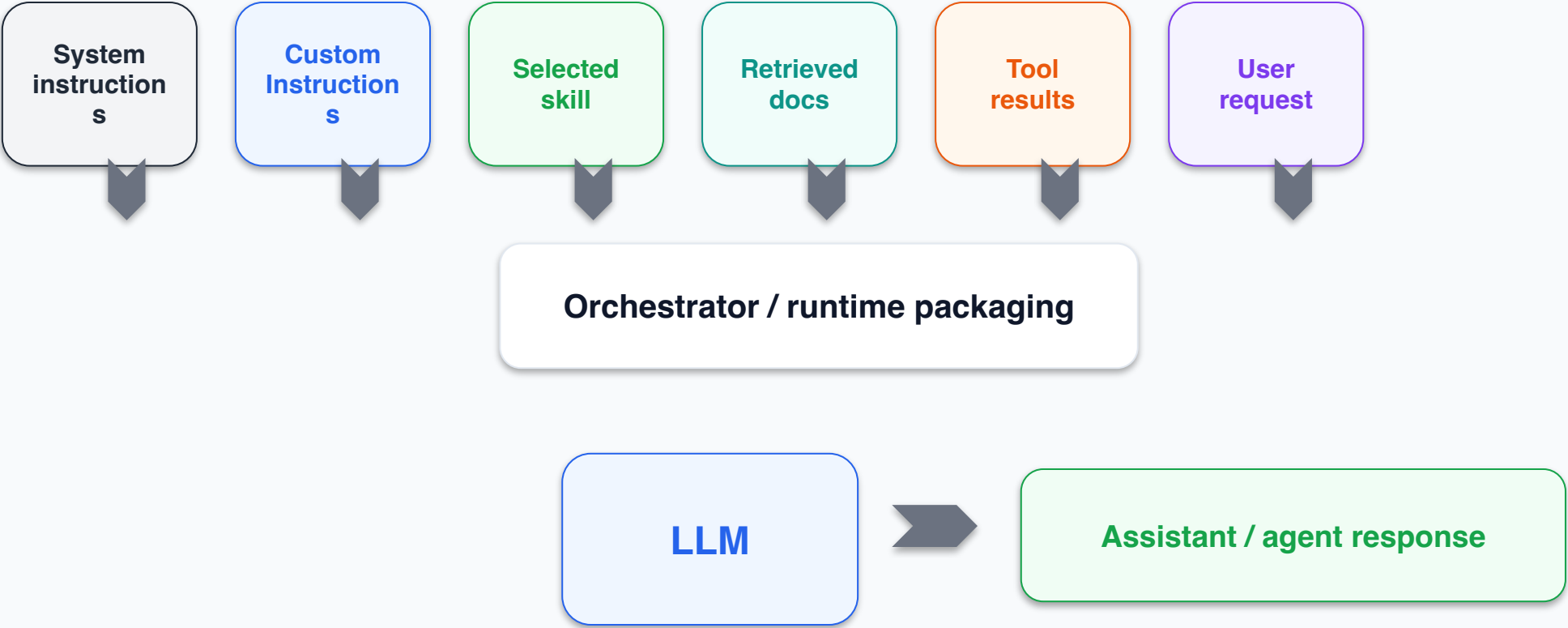
Only the current context is visible

Good outputs depend on sending the right information at the right time



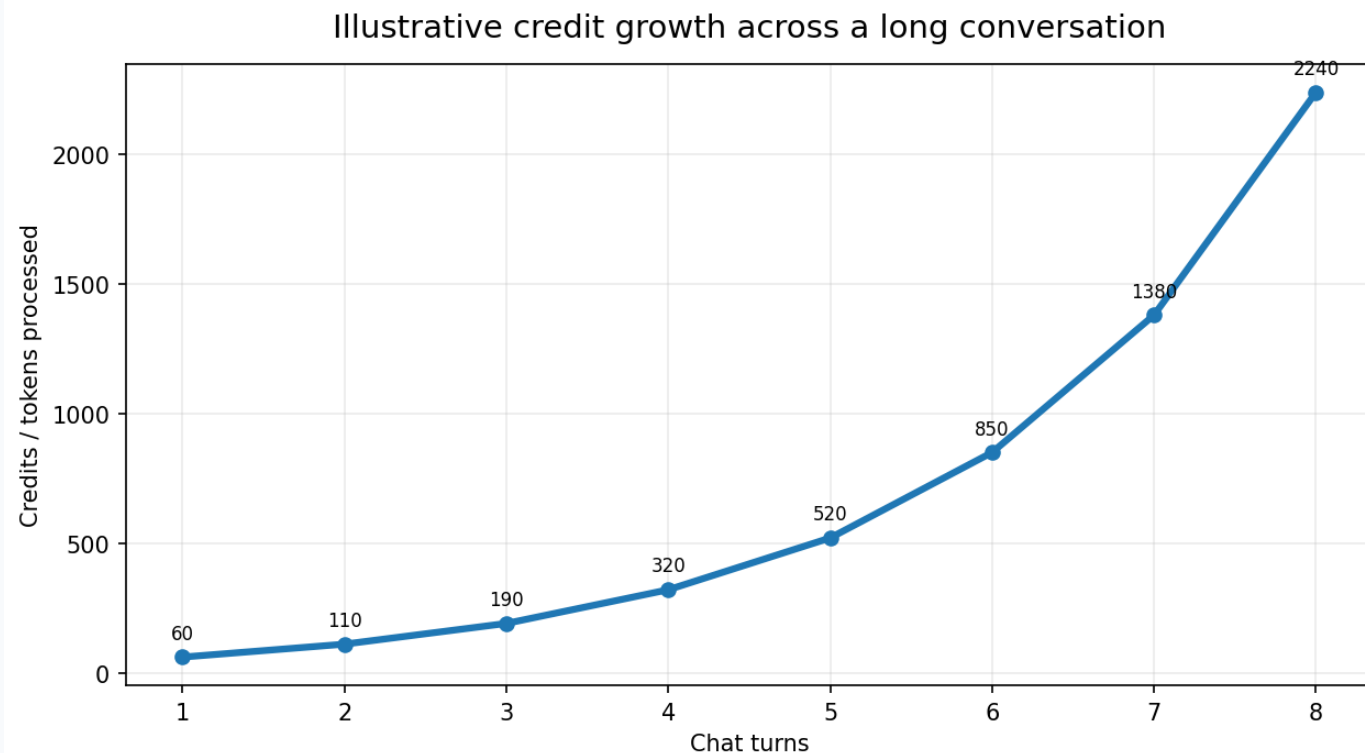
What the model actually sees

The model only sees the package assembled for the current request



Why credits can grow fast

Longer conversations can resend more context on every turn



What causes the jump?

- Each turn may resend instructions, history, and tool results.
- Retries and loops increase the total credits processed.
- Large context can grow cost faster than expected.

Takeaway

Keep conversations and context clean when possible.

Cost control patterns

Simple ways to keep agentic workflows efficient

Right model

Smaller context

Fewer retries

Clear instructions

Stop bad loops early

Use cheaper sub-agents

Golden rule

Use the smallest sufficient model with the smallest sufficient context.

Types of agents you can build

Examples across technical and business workflows

Developer agents

Architect
Software Engineer
QA / Test
DevOps
Security Review

Business agents

Meeting Assistant
Project Manager
Product / Intake
Priority / Routing
Support

Multimodal & domain agents

Document / PDF
Image / screenshot
Audio / transcript
Compliance / cost
Healthcare / operations

Demo focus

Engineering Manager Agent + Software Engineer Agent will be highlighted in the software workflow demo.

From transcript to business outputs

A multi-agent workflow for managers and non-technical users

Input

Meeting transcript or discussion notes

Transcript

Orchestrator

Specialist agents

Business outputs

Outputs

Summary • Decisions • Action items • Intake document • Priority list • Follow-up agenda

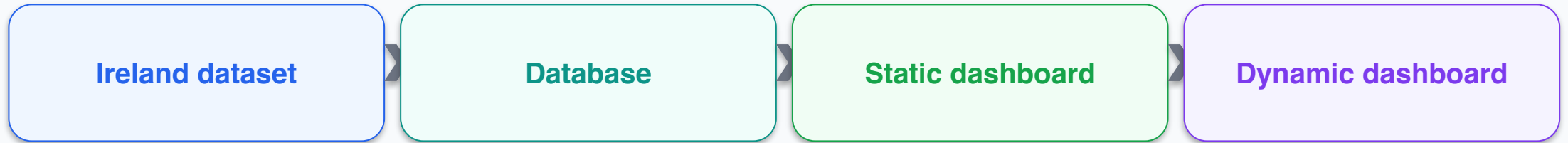
Manager workflow orchestration

How multiple agents collaborate inside the business demo



Application we will build

Ireland dataset → database → dashboards



Dataset input

Use an Ireland dataset as the starting point.

Build target

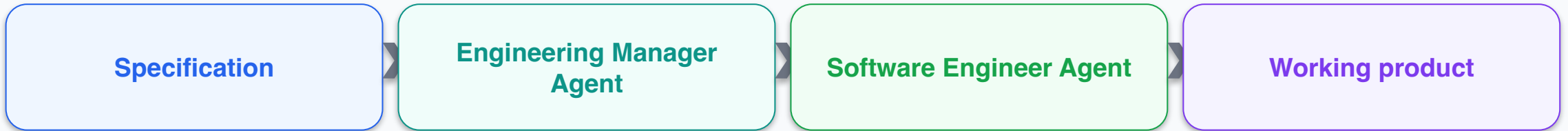
Load the data, structure it, and expose it through dashboards.

What this demonstrates

How agents help move from a requirement to a working software outcome.

Turning specification into a working software product

The software workflow highlighted in this workshop



Engineering Manager Agent

Clarifies the request
Frames the work
Breaks down tasks

Software Engineer Agent

Implements the solution
Builds DB + dashboards
Produces working outputs

Expected outcome

A more structured path from idea to product delivery.

Open-source tools & repositories

Useful examples to explore after the workshop

Spec Kit

An example of structured agentic workflow setup.

<https://github.com/github/spec-kit>

GSD

A reference point for agent-based workflow setup.

<https://github.com/gsd-build/get-shit-done>

Awesome Copilot

A collection of agent, prompt, and workflow ideas.

<https://github.com/github/awesome-copilot>

Skills

Examples of various skills patterns.

<https://www.skills.sh>

Use these as references, not prerequisites

We will point participants to these resources so they know what is available without trying to cover everything in one workshop.

Key takeaways

What participants should leave with

Workflow first — Design the process, not just the prompt.

Specialists help — Multiple agents can divide and clarify work.

Instructions matter — Files and skills make outputs more consistent.

Context drives quality — Send the right information at the right time.

Cost needs control — Model choice, context size, and retries all matter.

Thank you

Acknowledgements

Special thanks

David McVeigh
Optum / UHC tech leadership
Pauline Coll
Security team

With appreciation

Health & Safety team
Food & real estate
Volunteers: Zia, Srini & Soundar

And thank you

Attendees

For joining, engaging, and helping make the workshop valuable.